

Path Planning Accelerator with Rapidly-Exploring Random Tree



Tzu-Chieh Wei¹, Yi-Chung Wu²

¹Dept. of Electronics and Electrical Engineering, NYCU

²Institute of Electronics, NYCU

Research Overview

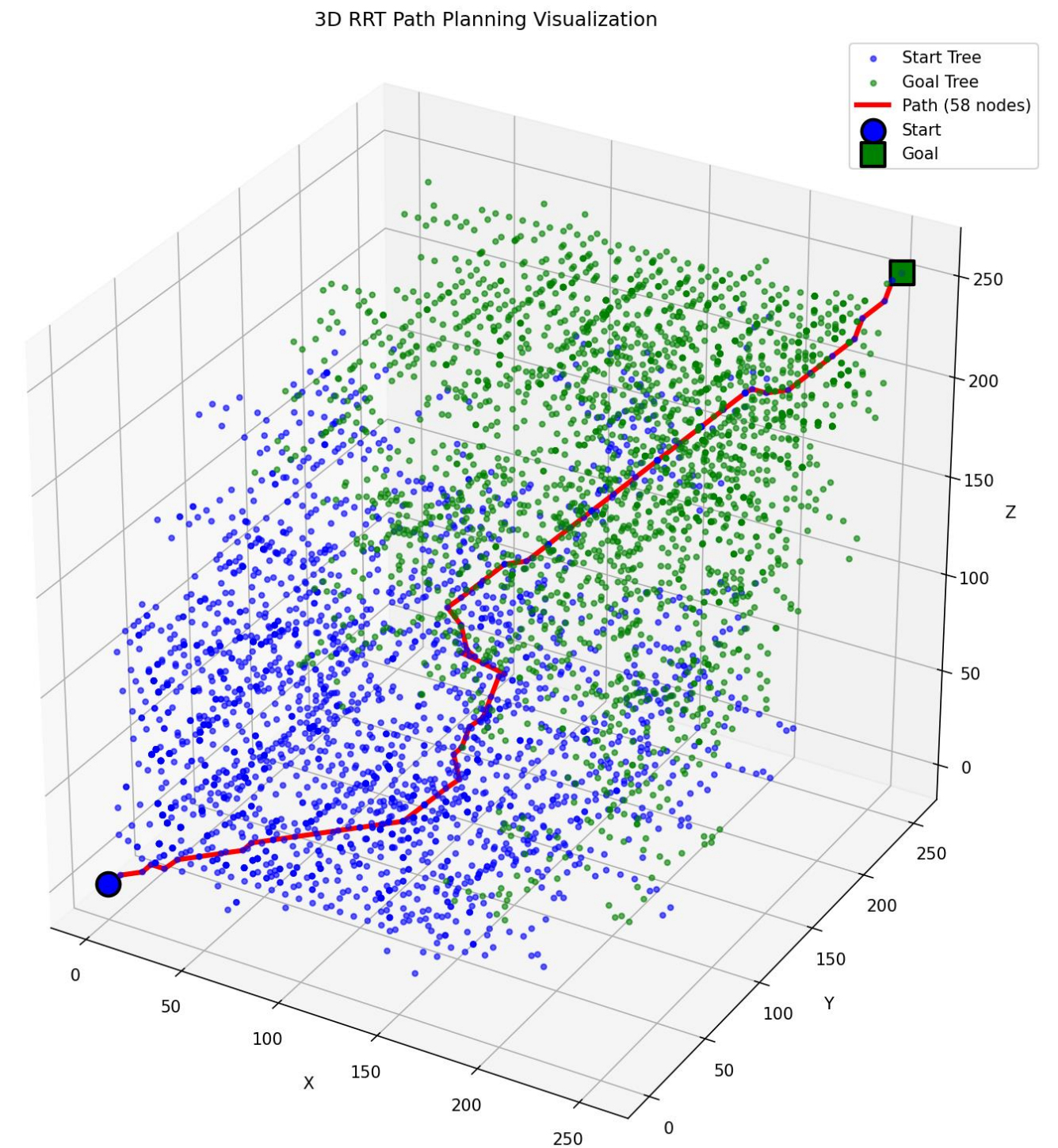
❖ Motivation

Real-time path planning is essential for autonomous vehicles operating in complex 2-D and 3-D environments. However, CPU-based software execution often suffers from high latency and large memory access overhead, especially as map resolution and dimensionality increase.

This work develops an RRT-based hardware accelerator to improve path-planning efficiency. By using dual-tree planning, parallel processing elements, and hardware-oriented memory organization, the proposed design aims to reduce search latency and evaluate the feasibility of low-cost real-time path planning.

❖ Key Techniques

- ❑ Grid map structure for tree memory and hybrid node searching.
- ❑ Hardware-optimized Manhattan distance and shift-and-compare vector.
- ❑ Boundary-aware connection checking to reduction of the checking latency.



Architecture and Algorithm

❖ RRT Algorithm and Optimization

```
function RRT (Random):
  NearestNode ← FindNearestNode(Random)
  VectorStep ← FindVector(NearestNode)
  if (VectorStep = (0, 0, 0))
    return
  NewNode ← NearestNode
  i ← 0
  while (i < 7)
    LastNode ← NewNode
    NewNode ← LastNode + VectorStep
    if (CollisionCheck(NewNode))
      break
    i++
  end
  if (i > 0)
    return LastNode
end
```

```
function ConnectionCheck (NewNode, Start):
  StartTree ← NewNode
  ReadGrid ← (NewNode.x, y, z >> 4)
  idx ← 0
  while (GridSize(ReadGrid) > idx)
    CandNode ← ReadGoalTree(ReadGrid)
    if (Distance(NewNode, CandNode) =
        (±1, ±1, ±1))
      return NewNode, CandNode
    idx++
  end
end
```

Grid Map Structure		
Node Coordinate	Parent Grid Coordinate	Parent Grid Index
Grid (0, 0, 0)	(0, 0, 0)	15
(0, 0, 0)	(0, 0, 0)	1
(8, 0, 8)	(0, 0, 0)	1
Grid (1, 0, 0)	(0, 0, 0)	2
(20, 4, 9)	(0, 0, 0)	2

◆ Nearest Node Distance Calculation

When calculating the nearest node, the conventional way is by L2 distance (Euclidean)

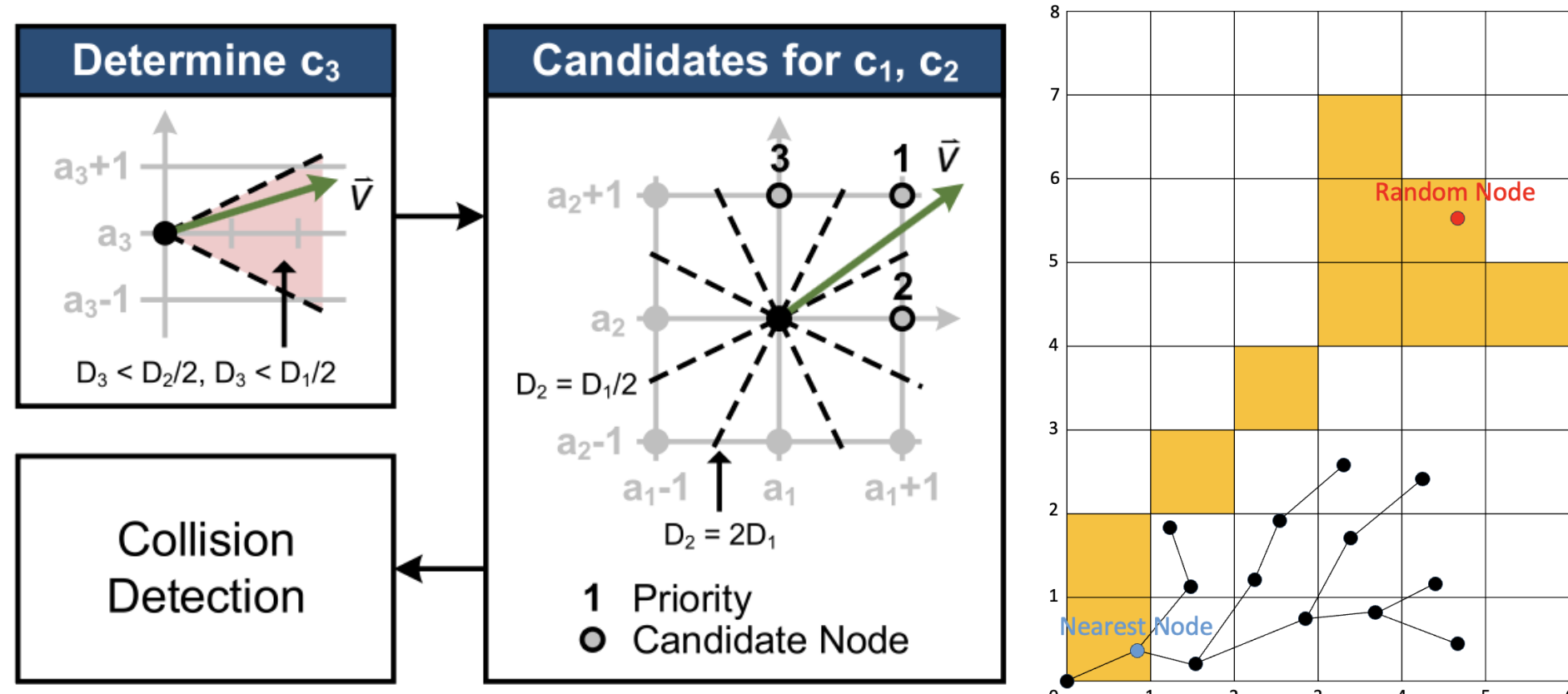
$$dist = \sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2}$$

To optimize the hardware computation, avoiding the multiplication and square root, the accelerator uses L1 distance (Manhattan)

$$dist = |\Delta x| + |\Delta y| + |\Delta z|$$

◆ Vector Step Generating Logic

Each node is placed on an integer grid point, so the extension direction is simplified to a discrete vector. For each axis, the movement is set to -1, 0, or +1 according to the relative position between the nearest node and the random sample.



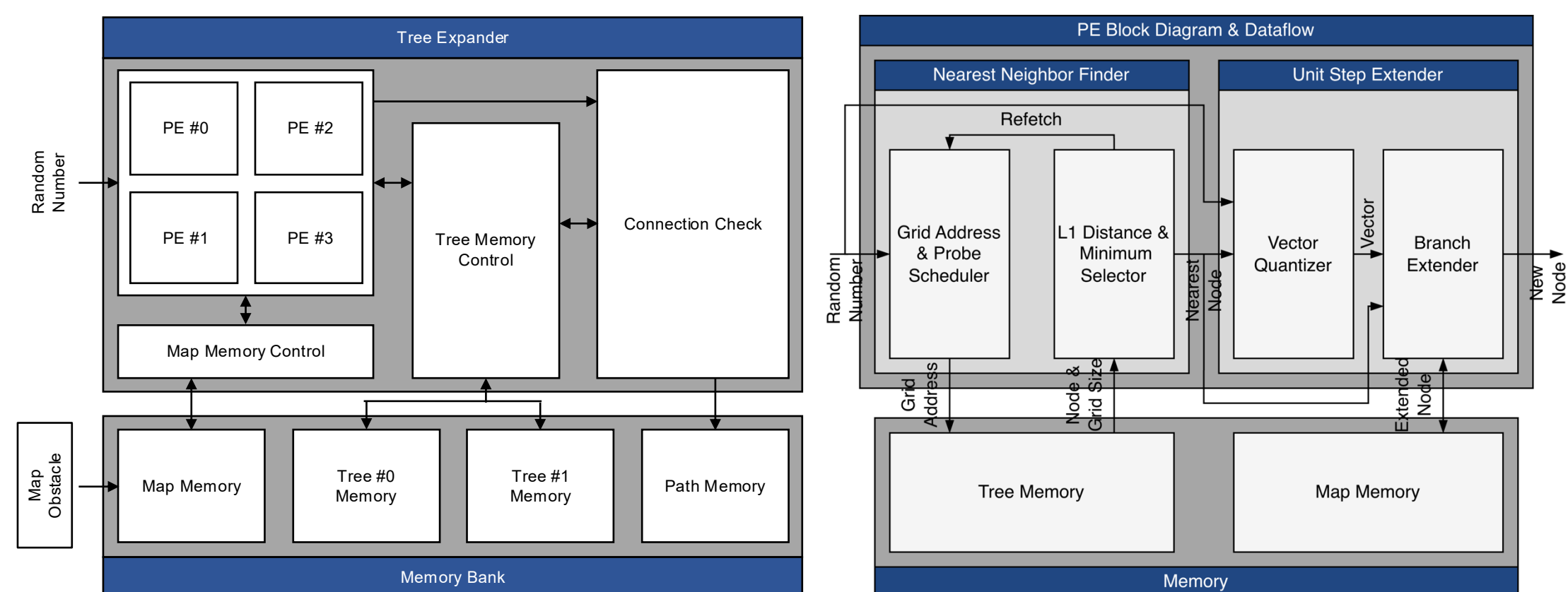
❖ Grid Map Structure

The 256³ 3-D map is divided into 2¹² grid cells, where each grid contains 16³ nodes. Tree nodes are stored according to their grid location, allowing the nearest-neighbor search to focus on local regions instead of scanning the entire tree.

❖ Hybrid Nearest Neighbor Search

A hybrid search strategy is used to find an approximate nearest node. This reduces memory access and lowers the search complexity from O(n) to approximately O(logN).

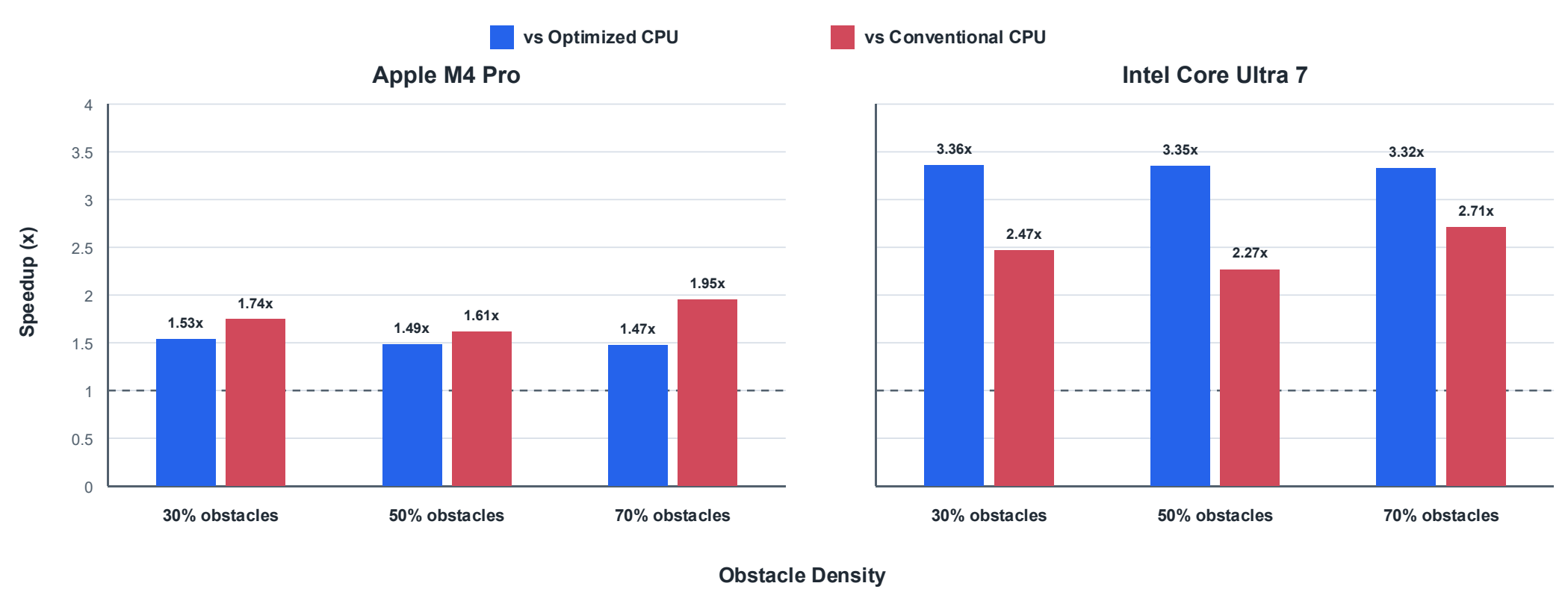
❖ System Architecture



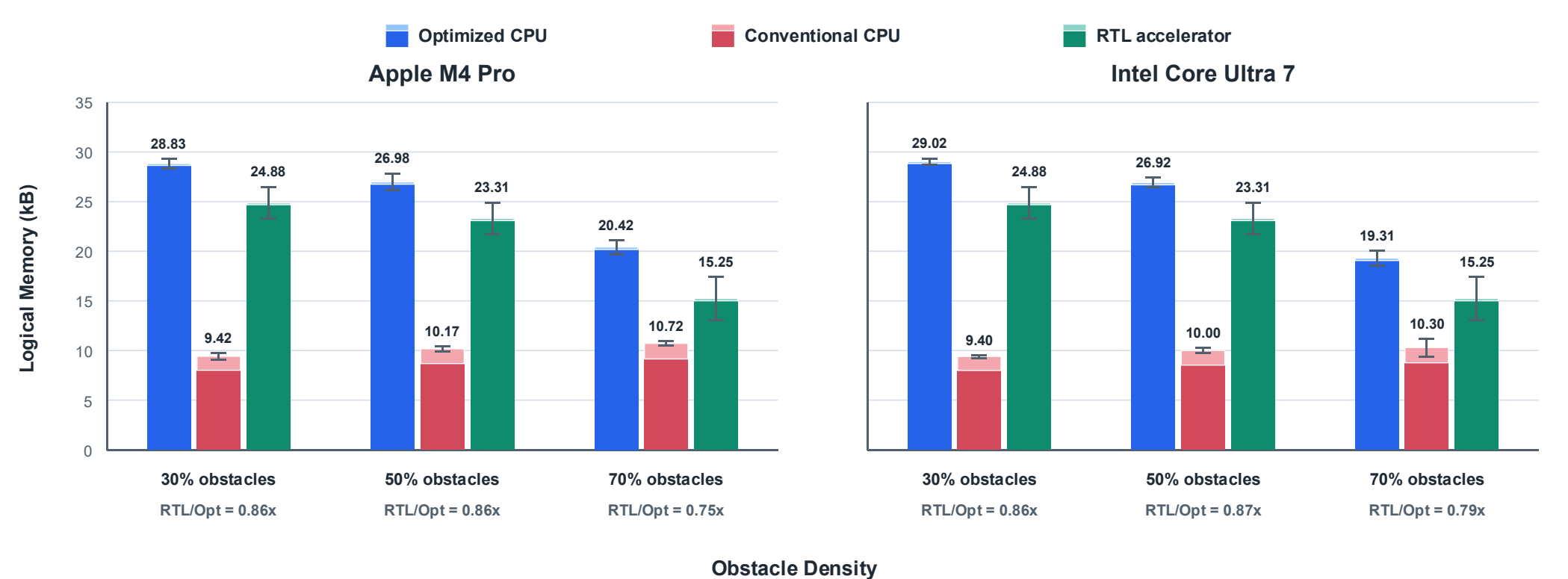
Experimental Results

❖ Optimized Performance

RTL Accelerator Speedup over CPU



Logical Tree + Path Memory Usage



❖ Post-Synthesis ASIC Implementation

Technology		TSMC 130nm
Area (μm^2)	Die	527.16 × 522.75
	Core	502.32 × 498.15
Frequency (MHz)		303.03
Map Size		256 ³
Mean Latency(ms/map)	30%	0.75
	50%	1.182
	70%	1.833

